

APTURA TECH SOLUTIONS
Python Development Internship – Batch 02

Week 01 Task Report

Smart Student Record Management System

Submitted By:	Hafiz Siddiqui
Program:	BS Software Engineering, SMIU Karachi
Internship Track:	Python Development Department
Task:	Task 01 – Smart Student Record Management System
Date:	July 15, 2026

1. Objective

Develop a strong understanding of core Python programming concepts through the practical implementation of a menu-driven **Smart Student Record Management System**, applying file handling for persistent storage and a manually implemented sorting algorithm for GPA-based ranking, followed by a technical evaluation of the system's scalability.

2. System Overview & Design

The system is a console-based, menu-driven application written in pure Python (no external frameworks) with the following core capabilities:

- **Add Student** – captures Roll No, Name, Department and GPA, with input validation (non-empty fields, GPA range 0.0–4.0) and duplicate Roll No prevention.
- **Update Student** – locates a record by Roll No and allows selective editing of any field, leaving a field unchanged if left blank.
- **Delete Student** – removes a record by Roll No after an explicit (y/n) confirmation prompt to prevent accidental data loss.
- **Search Student** – supports lookup by exact Roll No or partial, case-insensitive Name match, returning all matching records in tabular form.
- **Display All Records** – renders every record ranked by GPA, highest to lowest, using a custom-built Merge Sort (see Question 01).
- **File Handling** – all records are persisted to `students_data.json` using Python's built-in `json` module. The file is read once at startup and re-written after every Add/Update/Delete operation, so no data is lost between sessions.

Each student record is stored as a JSON object with four fields: `roll_no`, `name`, `department`, and `gpa`. The full, runnable source code is submitted alongside this report as `student_record_system.py`.

3. Question 01 – Custom Sorting Algorithm

Algorithm selected: Merge Sort (implemented from scratch – no built-in `sort()` or `sorted()` is used anywhere in the codebase). Records are ranked by GPA in descending order (highest to lowest).

3.1 How it works

Merge Sort is a divide-and-conquer algorithm. The list of student records is recursively split in half until each sub-list contains at most one element (which is trivially sorted). The sub-lists are then merged back together two at a time: at each merge step, the front elements of both sub-lists are compared, and the one with the higher GPA is appended to the result first. This continues until both sub-lists are fully consumed, producing a single sorted list.

3.2 Why Merge Sort was chosen

- **Guaranteed $O(n \log n)$ performance** in the best, average, *and* worst case. Simpler algorithms such as Bubble Sort, Selection Sort and Insertion Sort degrade to $O(n^2)$ as the number of students grows, which becomes noticeably slow once class sizes reach a few thousand. Quick Sort averages $O(n \log n)$ but can degrade to $O(n^2)$ on already-sorted or reverse-sorted input – a realistic case here, since students are frequently entered in roll-number order.
- **Stability** – Merge Sort preserves the relative order of records that share the same GPA (ties keep their original insertion order). This gives fair, predictable and reproducible class rankings, which an in-place, non-stable algorithm like standard Quick Sort or Heap Sort cannot guarantee.
- **Predictable, consistent behaviour** regardless of how the input data happens to be ordered, which matters for an internship-grade system that should behave the same way every time it is run.
- **Natural extension to large datasets** – because Merge Sort processes data in independent chunks that are merged together, the same divide-and-conquer idea extends directly to *external sorting* (sorting data that does not fit in memory), which becomes directly relevant when the system needs to scale (see Question 02).

3.3 Complexity comparison

Algorithm	Best Case	Average Case	Worst Case	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	No
Merge Sort (used)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes

3.4 Core implementation

```
def merge_sort_by_gpa(students):
    """Sorts student records by GPA, descending (highest first)."""
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort_by_gpa(students[:mid])
    right = merge_sort_by_gpa(students[mid:])
    return _merge(left, right)
def _merge(left, right):
```

```
merged, i, j = [], 0, 0
while i < len(left) and j < len(right):
    if left[i]["gpa"] >= right[j]["gpa"]: # ">=" keeps sort stable
        merged.append(left[i]); i += 1
    else:
        merged.append(right[j]); j += 1
merged.extend(left[i:])
merged.extend(right[j:])
return merged
```

4. Question 02 – Scaling to 100,000 Records

The current implementation is intentionally simple – a Python list held in memory, persisted to a single JSON file, with Roll No lookups done via a linear scan. This works well for a classroom-sized dataset, but several bottlenecks would appear well before 100,000 records:

- The entire file is read into memory and re-written on *every single* Add/Update/Delete, which becomes an $O(n)$ disk write for every operation.
- Search, Update and Delete all use a linear scan ($O(n)$) to find a record by Roll No.
- Display All re-sorts the *entire* dataset with every call, even if only the top few ranks are actually needed.

4.1 Proposed modifications

- **Replace the flat JSON file with a real database (e.g. SQLite for a single-machine app, or PostgreSQL/MySQL for a networked one).** A database engine handles buffered disk I/O, transactions, and concurrent access far more efficiently than rewriting an entire JSON file on every change.
- **Index the Roll No column** (as a PRIMARY KEY / B-tree index). This turns Search, Update and Delete from $O(n)$ linear scans into $O(\log n)$ index lookups – and with a hash index specifically, close to $O(1)$ average case. In pure Python without a database, the equivalent fix is to keep an in-memory dictionary keyed by `roll_no` (`{roll_no: student}`) alongside the list, giving $O(1)$ average lookups instead of scanning the whole list.
- **Avoid re-sorting the full dataset for every display.** If the UI only ever needs, say, the top 10 or top 50 ranked students, a **min-heap of fixed size k** gives those results in $O(n \log k)$ instead of $O(n \log n)$ for a full sort. If a fully ranked list is genuinely needed often, maintaining a persistently sorted index (updated incrementally on insert, e.g. via a balanced BST or a database index on GPA) avoids repeated full sorts entirely.
- **External / chunked sorting for memory-bound cases.** If 100,000+ records with several fields each grow too large to comfortably fit in memory, the Merge Sort approach from Question 01 extends naturally to an *external merge sort*: split the data into chunks small enough to sort in memory, write each sorted chunk to disk, then merge the chunks together – the same merge logic already implemented, just applied across files instead of in-memory lists.
- **Paginate reads instead of loading everything at once.** Both the file-read on startup and the Display All screen should fetch and show records in pages (e.g. 50–100 at a time via SQL LIMIT/OFFSET or cursor-based pagination) rather than materialising all 100,000 records in memory or printing them all to the console at once.
- **Batch and buffer writes.** Instead of rewriting the whole file per operation, use the database's native transaction/commit mechanism (or, without a DB, an append-only change log that is periodically compacted) so a single Add/Update/Delete is an $O(1)$ write rather than an $O(n)$ full-file rewrite.
- **Cache hot queries.** Frequently requested views – such as the current Top-10 by GPA – can be cached and only invalidated when a relevant record changes, avoiding recomputation on every request.
- **Handle concurrent access safely.** If multiple users/processes can modify records at the same time, rely on the database's transaction isolation (or explicit file locking, if staying file-based) to prevent race conditions and data corruption – something a single shared JSON file cannot safely guarantee.

In short: the algorithmic thinking behind Question 01 (choosing a reliable $O(n \log n)$ divide-and-conquer sort) is exactly the same thinking that scales the system up in Question 02 – replace $O(n)$ operations with

indexed/ $O(\log n)$ or $O(1)$ equivalents, avoid redundant full-dataset work, and let the storage layer (database) do the heavy lifting it is designed for.

5. Program Execution & Output Screenshots

The program was executed end-to-end in a live terminal session covering every menu option, including input validation and error handling. Captured terminal output for each stage follows.

```
hafiz@aptura-internship: ~/student_project

Welcome to the Smart Student Record Management System!
[INFO] Loaded 0 existing record(s) from 'students_data.json'.

=====
      SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit

Enter your choice (1-6): 9
[ERROR] Invalid choice. Please enter a number between 1 and 6.

=====
      SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit

Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-001
Enter Name: Ali Raza
Enter Department: Software Engineering
Enter GPA (0.0 - 4.0): 3.75
[SUCCESS] Student 'Ali Raza' (BSE-23F-001) added successfully.
```

Main menu, invalid-choice handling, and adding the first student record.

```
hafiz@aptura-internship: ~/student_project

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-002
Enter Name: Sara Khan
Enter Department: Computer Science
Enter GPA (0.0 - 4.0): 3.92
[SUCCESS] Student 'Sara Khan' (BSE-23F-002) added successfully.

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-003
Enter Name: Bilal Ahmed
Enter Department: Software Engineering
Enter GPA (0.0 - 4.0): 2.85
[SUCCESS] Student 'Bilal Ahmed' (BSE-23F-003) added successfully.

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
```

Adding the remaining student records (Sara Khan, Bilal Ahmed, Ayesha Tariq).


```
hafiz@aptura-internship: ~/student_project

=====
Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-004
Enter Name: Ayesha Tariq
Enter Department: Artificial Intelligence
Enter GPA (0.0 - 4.0): 3.92
[SUCCESS] Student 'Ayesha Tariq' (BSE-23F-004) added successfully.

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====
Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-005
Enter Name: Usman Sheikh
Enter Department: Data Science
Enter GPA (0.0 - 4.0): 3.10
[SUCCESS] Student 'Usman Sheikh' (BSE-23F-005) added successfully.

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====
Enter your choice (1-6): 1

--- ADD NEW STUDENT ---
Enter Roll No (e.g. BSE-23F-081): BSE-23F-001
[ERROR] A student with Roll No 'BSE-23F-001' already exists.
```

Adding the last student record and duplicate Roll No rejection when re-adding an existing student.

```
hafiz@aptura-internship: ~/student_project

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 5

--- ALL STUDENT RECORDS (Sorted by GPA: Highest -> Lowest) ---
Rank  Roll No      Name                Department           GPA
-----
1     BSE-23F-002    Sara Khan           Computer Science     3.92
2     BSE-23F-004    Ayesha Tariq       Artificial Intelligence 3.92
3     BSE-23F-001    Ali Raza           Software Engineering 3.75
4     BSE-23F-005    Usman Sheikh       Data Science         3.1
5     BSE-23F-003    Bilal Ahmed        Software Engineering 2.85
```

Display All Records – ranked by GPA (highest to lowest) using the custom Merge Sort. Note Sara Khan and Ayesha Tariq are tied at 3.92; the stable sort preserves their original entry order.

```
hafiz@aptura-internship: ~/student_project

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 4

--- SEARCH STUDENT ---
Search by: 1) Roll No    2) Name
Enter choice (1/2): 1
Enter Roll No: BSE-23F-002

1 record(s) found:
Roll No      Name                Department           GPA
-----
BSE-23F-002  Sara Khan           Computer Science     3.92
```

Search Student – lookup by Roll No.

```
hafiz@aptura-internship: ~/student_project

=====
      SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 2

--- UPDATE STUDENT ---
Enter Roll No of student to update: BSE-23F-003
Current details -> Name: Bilal Ahmed, Department: Software Engineering, GPA: 2.85
Leave a field blank to keep its current value.

New Name [Bilal Ahmed]:
New Department [Software Engineering]:
New GPA [2.85]: 3.50
[SUCCESS] Record for 'Bilal Ahmed' updated successfully.
```

Update Student – partial update (GPA changed, other fields left unchanged).

```
hafiz@aptura-internship: ~/student_project

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 3

--- DELETE STUDENT ---
Enter Roll No of student to delete: BSE-23F-005
Are you sure you want to delete 'Usman Sheikh' (BSE-23F-005)? (y/n): y
[SUCCESS] Student 'Usman Sheikh' deleted successfully.

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 5

--- ALL STUDENT RECORDS (Sorted by GPA: Highest -> Lowest) ---
Rank  Roll No      Name           Department      GPA
-----
1     BSE-23F-002    Sara Khan      Computer Science 3.92
2     BSE-23F-004    Ayesha Tariq   Artificial Intelligence 3.92
3     BSE-23F-001    Ali Raza       Software Engineering 3.75
4     BSE-23F-003    Bilal Ahmed    Software Engineering 3.5

=====
SMART STUDENT RECORD MANAGEMENT SYSTEM
=====

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Display All Records (Sorted by GPA)
6. Exit
=====

Enter your choice (1-6): 6

[INFO] All data saved. Exiting the system. Goodbye!
```

Delete Student with confirmation prompt, final re-ranked display, and clean exit.

6. Conclusion

This task reinforced core Python fundamentals – menu-driven program flow, input validation, JSON-based file handling, and CRUD logic – alongside a from-scratch implementation of Merge Sort and a structured evaluation of how the design would need to evolve for large-scale, production-grade data (indexing, databases, pagination, and external sorting). All deliverables – source code, this report, and execution screenshots – are submitted together for Week 01 of the Python Development track.